Министерство науки и высшего образования РФ

Пензенский государственный университет

Кафедра «Вычислительная техника»

ОТЧЁТ

По лабораторной работе №3

По дисциплине «Логика и основы алгоритмизации в инженерных задачах»

На тему «Динамические списки.»

Выполнили студенты группы 25ВВВ3:

Сергеева С.М.

Иванова С.А.

Приняли:

к.т.н., доцент Юрова О. В.

к.т.н., доцент Деев М. В.

Пенза 2026

Цель: Изучить динамические структуры данных на примере односвязного линейного списка и научиться реализовывать на основе односвязного списка три классические структуры данных - приоритетную очередь, очередь (FIFO) и стек (LIFO).

Ход работы.

Задание 1: Реализовать приоритетную очередь, путём добавления элемента в список в соответствии с приоритетом объекта (т.е. объект с большим приоритетом становится перед объектом с меньшим приоритетом).

Описание программы:

Программа реализует приоритетную очередь на односвязном динамическом списке. Узел описывается структурой struct node с полями inf[256] (имя объекта), priority (приоритет) и next (указатель на следующий элемент).

Глобальные указатели head и last отмечают начало и конец списка, f используется для поиска, dlinna хранит длину. Список постоянно поддерживается упорядоченным по возрастанию priority.

Функция get_struct запрашивает имя и приоритет, выделяет память через malloc и возвращает новый узел.

Функция push вставляет элемент по приоритету: если список пуст или приоритет нового узла меньше приоритета головы, узел становится новой головой; иначе список просматривается до места, где следующий узел имеет приоритет больше вставляемого, и новый элемент ставится перед ним. Нестрогое условие curr->next->priority <= p->priority обеспечивает FIFO для равных приоритетов.

Функция pop извлекает элемент из головы и освобождает память. Функции print_list, find и del выводят, ищут и удаляют элементы по имени, корректно обновляя head, last и dlinna.

Вставка имеет сложность O(n) из-за поиска места, извлечение — O(1). Работа идёт через меню с очисткой экрана и паузой после операции.

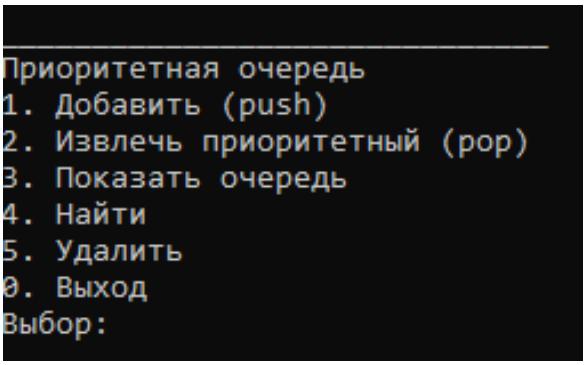


Рисунок 1 – Меню первой программы.

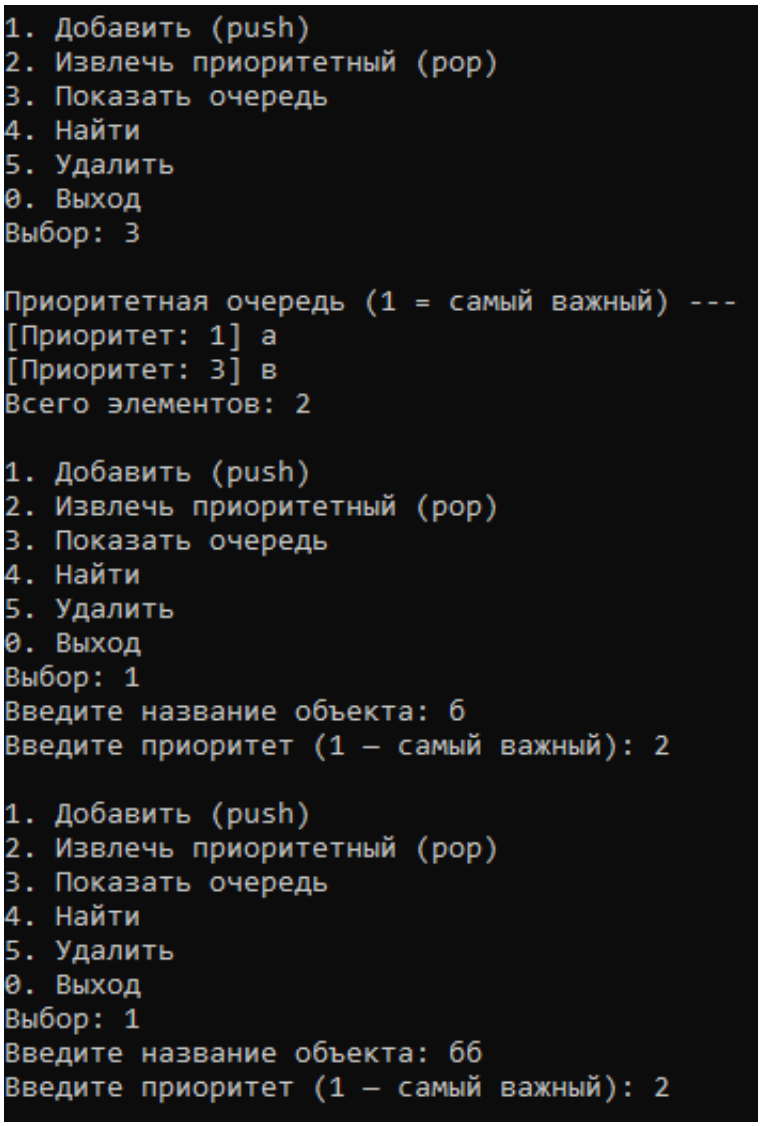


Рисунок 2 – Приоритетная очередь из 2-х элементов.

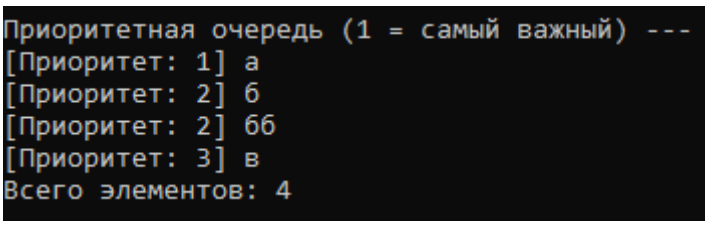


Рисунок 3 – Приоритетная очередь из 4-х элементов.

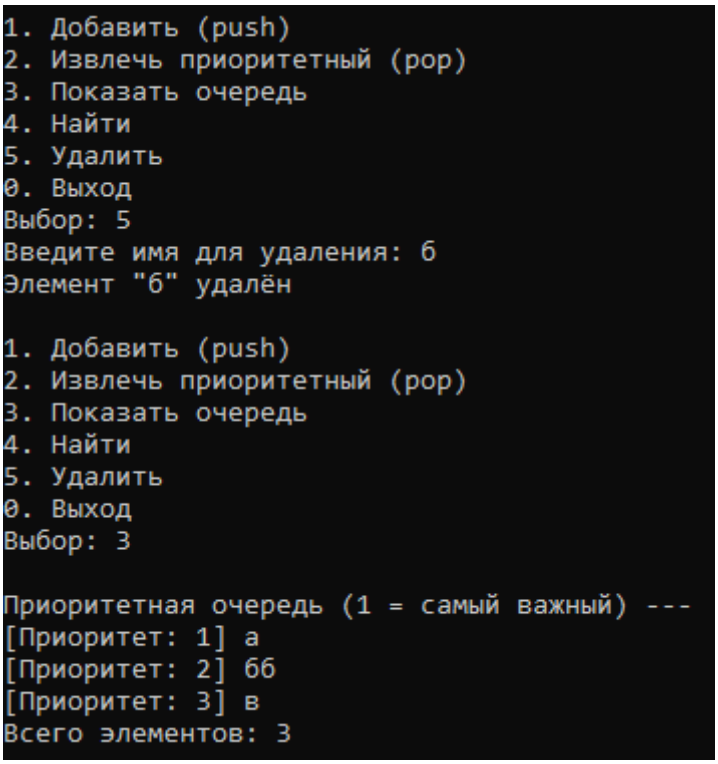


Рисунок 4 – Удаление элемента.

Задание 2: На основе приведенного кода реализовать структуру данных Очередь.

Описание программы:

Программа реализует очередь по принципу «первым пришёл, первым вышел». Структура без поля priority. Указатель head отмечает начало, last — конец, f служит для поиска, dlinna хранит длину.

Отличие от приоритетной очереди только в теле функции push: новый элемент всегда добавляется в конец. Если список пуст, узел становится одновременно головой и хвостом; иначе он прикрепляется к текущему хвосту через last->next, после чего last переносится на новый узел.

Такой способ не требует перебора списка и выполняется за O(1).

Функция pop извлекает элемент из головы, то есть удаляет самый ранний по времени поступления узел, и освобождает память; если очередь становится пустой, last обнуляется.

Остальные функции — get_struct, print_list, find, del — совпадают с первой программой.

Обе основные операции очереди имеют сложность O(1), а порядок извлечения совпадает с порядком поступления.

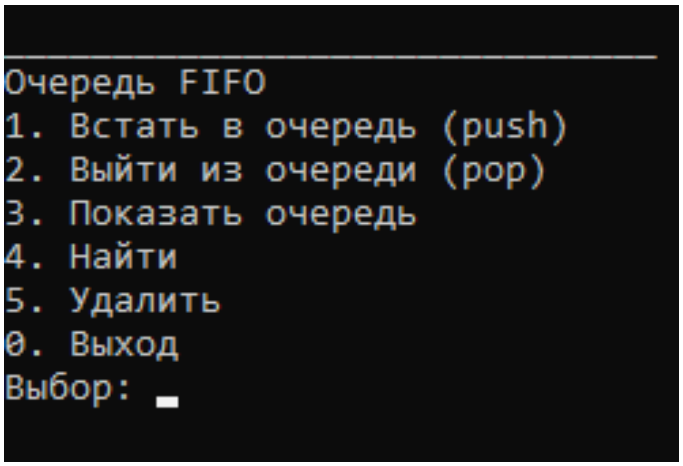


Рисунок 5 – Меню второй программы.

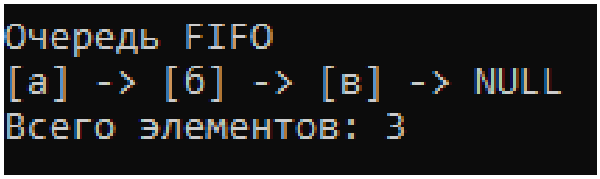


Рисунок 6 – Очередь из 3-х элементов.

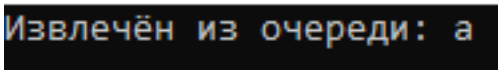


Рисунок 7 – Извлечение элемента.

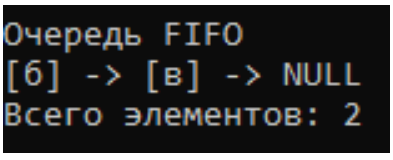


Рисунок 8 – Очередь из 2-х элементов.

Задание 3: На основе приведенного кода реализовать структуру данных Стек.

Описание программы:

Программа реализует стек по принципу «последним положил, первым забрал». Узел имеет те же поля, что и в очереди. Используется только указатель head, отмечающий вершину стека, указатель f для поиска и счётчик dlinna; last не нужен, поскольку доступ идёт только через вершину.

Отличие в функции push: новый узел вставляется в начало списка - его next направляется на текущую голову, а head перемещается на новый узел.

Вставка выполняется за O(1).

Функция pop снимает элемент с вершины, то есть из головы, и освобождает память.

Поскольку добавление и извлечение происходят с одного конца, стек обеспечивает обратный порядок обработки. Остальные функции совпадают с предыдущими программами, меняется только тело push. Функция print_list выводит содержимое стека сверху вниз.

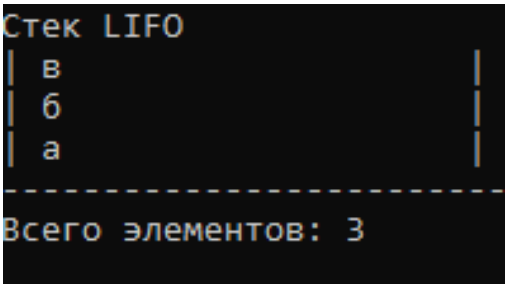


Рисунок 9 – Стек из 3-х элементов.

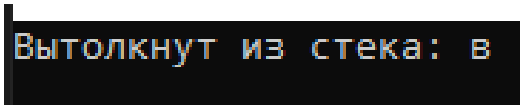


Рисунок 10 – Извлечение элемента.

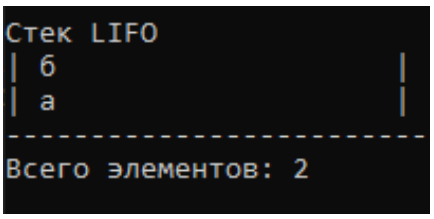


Рисунок 11 – Стек из 2-х элементов.

Вывод: В ходе лабораторной работы были изучены принципы построения динамических структур данных на примере линейного списка и реализованы три классические структуры: приоритетная очередь, очередь (FIFO) и стек (LIFO). Все три используют один и тот же узел и один и тот же набор функций, а различаются только телом вставки push: в стеке в начало, в очереди в конец, в приоритетной очереди - упорядочено по возрастанию приоритета. Извлечение везде идёт из головы за O(1). Это показывает, что приоритетная очередь, очередь и стек - частные случаи линейного списка, отличающиеся только способом вставки.